

Binary Heap

Serve per implementare le Code con priorità

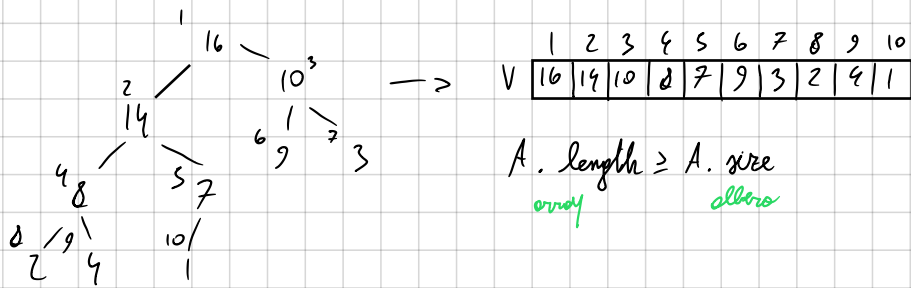
Questa struttura serve per implementare l'algoritmo di ordinamento Heap Sort

Cos'è? : È una struttura dati memorizzata come un array ma immaginata come albero binario quasi completo

Cosa vuol dire quasi completo? : 1) Tutti i livelli tranne al più l'ultimo sono completi

2) L'ultimo livello è riempito da sinistra a destra

+ Proprietà dello heap (tipizzate dopo)



Dato un nodo i otteniamo sapere dire: 1) $\text{parent}(i) = i/2$

2) $\text{left}(i) = 2i$

3) $\text{right}(i) = 2i + 1$

altezza nodo i : Lunghezza del più lungo cammino da i a una delle foglie dell'albero radicato in i

altezza heap : Altezza delle radici

N. nodi max di un heap di altezza h : $2^{h+1} - 1$

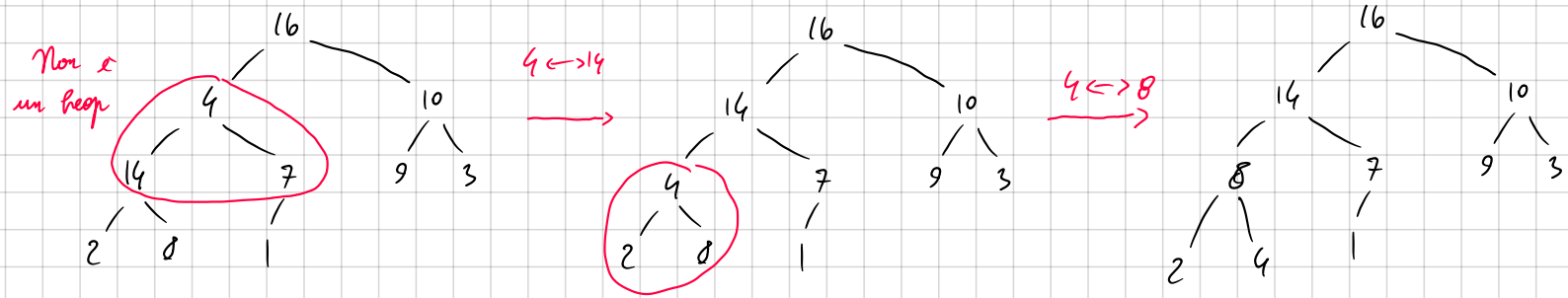
Quanti nodi al min? : 2^h

L'altezza di un heap binario? : $h = \log_2 n$

Proprietà dello heap : Tutti i nodi tranne le radici devono rispettare le seguenti proprietà :

1 - Max Binary Heap) $A[\text{parent}(i)] \geq A[i]$ $\rightarrow$ Le radici è il Max

2 - Min Binary Heap) $A[\text{parent}(i)] < A[i]$ $\rightarrow$ Le radici è il minimo



procedure **MAX-Heapify**(A, i)

$l = \text{left}(i)$

$r = \text{right}(i)$

$\text{largest} = i$

if $l \leq A.\text{heapsize}$ AND $A[l] > A[\text{largest}]$ then
 $\text{largest} = l$

if $r \leq A.\text{heapsize}$ AND $A[r] > A[\text{largest}]$ then
 $\text{largest} = r$

if $i \neq \text{largest}$ then
 swap $A[i]$ e $A[\text{largest}]$
MAX-Heapify($A, \text{largest}$)

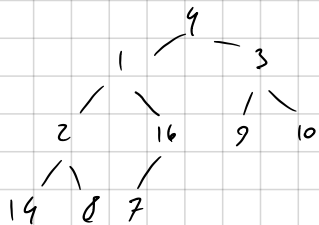
$\Theta(1)$

Caso migliore $\Rightarrow i = \text{largest} = \Theta(1)$

Caso peggiore $\Rightarrow i \neq \text{largest} \Rightarrow T(n) = \Theta(1) + T(n-1) = \Theta(n)$

Come creare un heap partendo da un array

$A = 4 \ 1 \ 3 \ 2 \ 16 \ 9 \ 10 \ 14 \ 8 \ 7$



n è l'altezza del
 nodo corrente

procedure BuildMaxHeap (A)

A.heapsize = A.length

for $i = \overset{\text{ultimo nodo}}{A.length/2}$ down to 1

MaxHeapify (A, i)] $T_H(h) = O(h)$

$$h = \log_2 n$$

$$T_{\text{Build}}(n) = \frac{n}{2} \cdot T_H(h) = \frac{n}{2} \cdot T_H(\log_2 n) = \frac{n}{2} O(\log n) = O(n \log n)$$

Costo pessimistico
si può fare di meglio

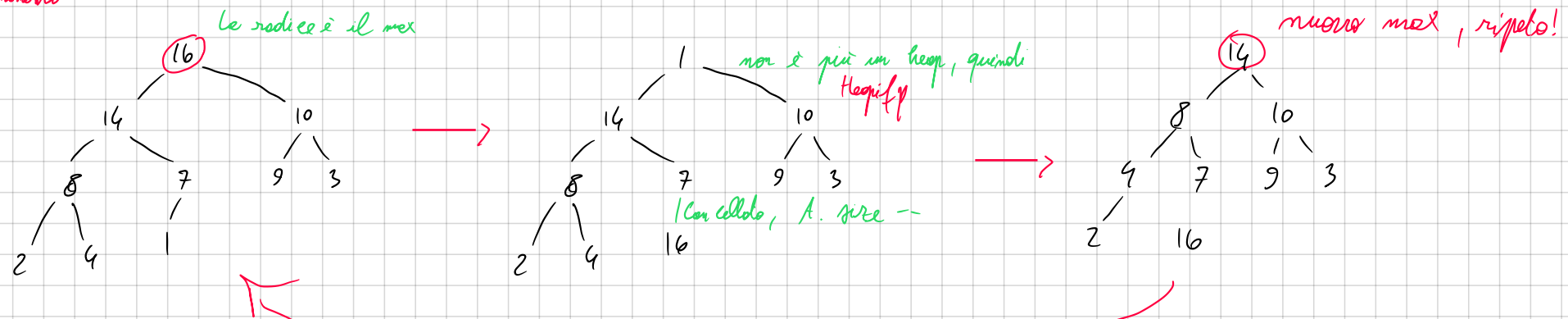
$$T_{\text{Build}} = \frac{n}{2} T_H(0) + \frac{n}{2^2} T_H(1) + \frac{n}{2^3} T_H(2) + \dots + 1 T_H(\log_2 n) = \sum_{i=0}^{\log n} \frac{n}{2^{i+1}} \cdot T_H(i) = \sum_{i=0}^{\log n} \frac{n}{2^{i+1}} \cdot c \cdot i = cn \cdot \sum_{i=0}^{\log n} \frac{i}{2^{i+1}} = \Theta(n)$$

↓
converge
 $\Theta(1)$

Chiamate heapify $h=1 = \frac{n}{2} - \frac{n}{4} = \frac{n}{4} = \frac{n}{2^2}$

Chiamate heapify $h=2 = \frac{n}{4} - \frac{n}{8} = \frac{n}{8} = \frac{n}{2^3}$

Ordinamento



procedure **heapsort** (A)

Build Max Heap (A)

for $i = A.length$ down to 2 do
 scambia $A[i]$ e $A[A.heapsize]$

$A.heapsize = A.heapsize - 1$

 Max Heapify (A, i)

in-place ? : Si

stabile ? : No

Code con priorità

cos'è: Un tipo di dato astratto

Operazioni disponibili:

- $\text{Insert}(S, x)$

- $\text{Extract Max}(S) \rightarrow x$

- $\text{Increase Key}(S, x, K)$

↑
nuova chiave
 $K \geq x$

procedure $\text{HeapExtractMax}(S)$

if $S.\text{heapsize} = 0$ then error "underflow"

$\text{max} = S[1]$

exchange $S[1]$ con $S[S.\text{heapsize}]$

$\text{MaxHeapify}(S, 1)$

return max

$\Theta(1)$

procedure $\text{HeapIncreaseKey}(S, x, K)$

if $S[x] > K$ then error "Key is smaller"

$S[x] = K$

While $x > 1$ and $S[\text{parent}(x)] < S[x]$ do

exchange $S[\text{parent}(x)]$ con $S[x]$

$x = \text{parent}(x)$

$\Theta(\log n)$

procedure $\text{HeapInsert}(S, K)$

if $S.\text{length} = S.\text{heapsize}$ then error "overflow"

$S.\text{heapsize} = S.\text{heapsize} + 1$

$S.\text{heapsize} = -\infty$

$\text{HeapIncrease}(S, S.\text{heapsize}, K)$